

“案例库 + 决策器”的机制对比与优化方向

——以“智能体合成钻石镐”为贯穿实例，从开放世界长序任务特点出发——

核心问题：我的案例库 + 决策器与现有方法的机制区别在哪里？

开放世界长序任务的哪些特点，让这套设计具备别的方法做不到的优势？

如何针对性优化，使创新点扛得住审稿人的拷打？

配套系统：XENON-plus（状态化案例库 + RADS 检索增强决策器 + 规则门控 Option 技能调度）

报告日期：2026 年 6 月 11 日

目录

一、本报告的切入点：把对比锁定在“同一个决策瞬间”	3
二、现有方法在决策点 D 各自怎么运转	3
2.1 检索注入类 (ExpeL / RAP / TRAD)：经验进了 prompt，信不信交给大模型	3
2.2 经典 CBR 类 (DS-Agent / 安全驾驶 CBR-LLM)：四步循环，决策寄生在“修正”里	3
2.3 过程验证器类 (PAV / AgentPRM)：会打分，但不连案例库、不会弃权	4
2.4 Minecraft 记忆类 (Optimus-1 / JARVIS-1 / 基线 XENON)：案例库齐全，决策器缺位	4
三、我的方法在决策点 D 怎么运转（逐步拆解）	4
3.1 第一步：把“当前状态”编码成结构化查询，而不是只看子目标名	4
3.2 第二步：用注意力从案例库检索“同类状态”的历史证据	5
3.3 第三步：对每个候选动作输出条件成功概率，并叠加一个稳健先验	5
3.4 第四步：低置信时弃权，把决策权交给规划器	5
3.5 低层：当问题是“环境卡住”而非“选错动作”，Option 调度器接管	6
3.6 决策点 D 上的并排对照	6
四、为什么是“开放世界长序任务”让我的设计拿到独有优势	7
4.1 特点一：奖励稀疏且延迟——只有任务终点才有信号	7
4.2 特点二：状态高维且地形多变——“看起来相似”常常“实则不可迁移”	7
4.3 特点三：失败高频且多为“环境卡死”——错因常在执行不在决策	7
4.4 特点四：分布外情形不可避免——开放世界永远有没见过的局面	7
4.5 特点五：高风险动作不可逆——掉进岩浆就重来	7
五、我的方法的不足，与针对性优化方案	8
5.1 优化一（最该先做）：证明“案例库环”不是摆设——检索消融	8
5.2 优化二：把弃权从“固定阈值”升级为“有保证的选择性决策”	8
5.3 优化三：把 state-delta 锚到 PAV 的“进展”理论，并做对照	9
5.4 优化四（更前沿）：让决策器反过来驱动案例采集——主动经验闭环	9
5.5 优化五：补齐与“隐式复用”的正面对照基线	9
5.6 优化方案与对应实验一览	9
六、可写进论文的“区分性陈述”与审稿问答预案	10
6.1 一句话区分性陈述（建议放进 intro 与 related work 收尾）	10
6.2 审稿人最可能的六个问题与应答	10
七、结语：把“组合优势”讲成“必须如此组合的理由”	11

一、本报告的切入点：把对比锁定在“同一个决策瞬间”

评价“案例库 + 决策器”这类方法，最容易流于泛泛——大家都说自己能“复用经验、辅助决策”。要把区别讲到机制层，必须把所有方法摁在同一个具体决策瞬间上，看它们各自吐出什么、依据什么、错了怎么办。本报告自始至终使用同一个实例：智能体正在执行长序任务“合成一把钻石镐”（`diamond_pickaxe`），这是 Minecraft 智能体研究的经典长程任务，约需依次完成十余个子目标（砍木→做木板→做木镐→挖石→做石镐→下矿→挖铁→熔铁→做铁镐→深入挖钻→合成钻石镐）。

我们把镜头停在其中一个高风险瞬间，称为决策点 D：

决策点 D（贯穿全文的统一情境）

智能体已持有铁镐，库存中有 2 颗钻石、若干铁锭，当前 y 坐标 = 11（深岩层、岩浆与空洞高发），当前子目标 `waypoint` = “mine_diamond（再挖 1 颗钻石以凑齐 3 颗）”。候选高层动作有若干个，例如：①继续水平挖掘 `explore_horizontal`；②向下挖 `dig_down`；③返回安全区 `retreat`。

问题：此刻该选哪个动作？更关键的是——智能体凭什么相信自己的选择，万一历史经验在这个具体状态下并不可靠，它知不知道？

围绕决策点 D，第二章先讲清现有方法在这一刻各自怎么运转（CBR 类、检索注入类、过程验证器类、Minecraft 记忆类），第三章讲我的方法在这一刻怎么运转、机制差异是什么，第四章从开放世界长序任务的五个结构性特点出发，论证为什么我的设计能拿到别人拿不到的优势、又在哪里仍有不足，第五章给出针对性优化方案与对应的消融实验，第六章直接给出可写进论文的“区分性陈述”与审稿人问答预案。

二、现有方法在决策点 D 各自怎么运转

2.1 检索注入类（ExpeL / RAP / TRAD）：经验进了 prompt，信不信交给大模型

以 ExpeL (AAAI 2024) 为代表。在决策点 D，它的流程是：把当前局面编码成查询，从经验库里检索若干条最相似的历史轨迹片段（比如“上次在 y=12 挖钻时水平挖掘成功了”），连同事先蒸馏的自然语言洞见（“深层挖矿注意脚下”）一起塞进大模型的上下文，然后让大模型自己读着这些材料决定选哪个动作。RAP（检索增强规划）更进一步，让模型显式生成“检索键”再去取经验；TRAD (SIGIR 2024) 则把检索粒度对齐到当前这一步而非整条轨迹。

这一类的机制特征：检索是显式的，但“这条经验在 y=11 这个具体状态下到底可不可信、该信几分”没有任何量化——完全由大模型在生成时隐式权衡。后果有两个：其一，无法离线评估它的复用判断准不准（你没法对“大模型读了经验后的隐式信心”算 AUC）；其二，没有弃权出口——大模型读完经验总会给一个动作，哪怕检索到的经验其实来自完全不同的地质环境（比如那条成功经验其实发生在没有岩浆的平地矿洞）。在长序任务里，这种“看起来相似、实则不可迁移”的经验恰恰是致命的。

2.2 经典 CBR 类（DS-Agent / 安全驾驶 CBR-LLM）：四步循环，决策寄生在“修正”里

DS-Agent (ICML 2024) 把经典案例推理的 `retrieve`→`reuse`→`revise`→`retain` 四步搬到大模型上。若把它的范式映射到决策点 D：`retrieve` 按相似度取一条最像的历史案例，`reuse` 直接套用其方案，`revise` 由大模型根据执行反馈改写方案，`retain` 把新案例存回库。它的标签来自外部可验证信号（在数据科学里是排行榜分数），这一点和我的环境校验同源、是它的优点。

但它的“决策器”是隐式的、寄生在 revise 环里的——由大模型自由改写，没有一个独立模块量化“这条 case 的效用有多高、该不该采纳”。安全攸关驾驶的 CBR-LLM (Safety Science 2026) 则把案例当作“安全护栏”：用检索到的事故案例约束大模型不要做出危险决策。映射到 D 点，它相当于用历史教训兜住大模型，但护栏与决策依据揉在一起，没有把“生成合法候选”与“在候选内排序”分开。

2.3 过程验证器类 (PAV / AgentPRM)：会打分，但不连案例库、不会弃权

这一类是“决策器”概念在推理领域最成熟的形态。过程奖励模型 (PRM) 会为每个候选步骤输出一个质量分数用于重排序。PAV (ICLR 2025 Spotlight) 给出了一个深刻论点：步骤的价值不应是它的绝对正确性，而应是它带来的“进展”——执行这一步前后最终成功概率的增量。AgentPRM 把这套搬进交互式智能体。

映射到决策点 D：一个智能体 PRM 会给 explore_horizontal / dig_down / retreat 各打一个分（“这个动作让我离凑齐 3 颗钻石更近的程度”），选分高的。这已经非常接近我的决策器在做的事。但三个机制缺口很关键：第一，它不检索历史案例，打分不附带证据，你无法追问“凭哪几次历史经验给的这个分”；第二，它没有弃权通道——分数低也得在这几个候选里选一个，不会切换到“放弃复用经验、回到从头规划”；第三，也是对长序具身任务最要命的一点，它估“进展”（成功概率增量）靠的是蒙特卡洛推演——从当前步采样很多条续写看最终成功率。在 Minecraft 里采样一条到任务终点的续写代价极高（要真的把游戏跑到底），这条路几乎走不通。

2.4 Minecraft 记忆类 (Optimus-1 / JARVIS-1 / 基线 XENON)：案例库齐全，决策器缺位

这是我在领域内最直接的邻居。Optimus-1 (NeurIPS 2024) 有分层知识图谱加抽象多模态经验池；JARVIS-1 (TPAMI 2024) 存（任务、计划、多模态状态）。映射到决策点 D：它们检索“mine_diamond 这个子目标以前是怎么做成的”，把相似经验喂给多模态大模型，由模型隐式决定动作。它们的记忆单元是任务级的，回答的是“这个任务以前怎么做成的”。

我的基线 XENON (ICLR 2026) 更具体：它为每个 waypoint 维护一张动作计数表，记录“mine_diamond 下 explore_horizontal 成功过 5 次、失败 2 次”，决策规则是“有成功经验就选净成功次数最高的动作”。这已经有了“决策器”的雏形，但它是一个无状态条件、无置信、无弃权的计数阈值——它不区分“y=11 带岩浆”和“y=40 安全矿洞”，只要 waypoint 都是 mine_diamond，计数就混在一起。这正是我要超越的起点。

三、我的方法在决策点 D 怎么运转（逐步拆解）

XENON-plus 在决策点 D 走的是两级流程：高层由检索增强决策器 RADS 仲裁普通高层动作，低层在需要环境恢复时由规则门控的 Option 调度器接管。先看高层。

3.1 第一步：把“当前状态”编码成结构化查询，而不是只看子目标名

RADS 不像计数表那样只认 waypoint 名字。它把决策点 D 的状态编码成一个 52 维结构化特征：当前装备 (iron_pickaxe)、所处生物群系、waypoint 序号与在整条任务中的位置、y 坐标归一化值与分桶 (y=11 落入“<15 深岩层”这一危险桶)、关键库存计数 (含 diamond=2、iron_ingot、cobblestone 等 13 项)、工具持有标志、库存种类数。再加上 waypoint、final_goal、候选动作三个类别型字段各自的嵌入。

机制差异点 ①：决策对象从“子目标”细化到“状态条件”

计数表 (XENON) 看到的是“mine_diamond → 选历史最优动作”，它把 $y=11$ 带岩浆和 $y=40$ 安全矿洞当成同一种情况。

RADS 看到的是“mine_diamond 且 $y \in$ 深岩桶 且 已持铁镐 且 diamond=2”这一具体状态向量，它能区分前述两种地质环境，因为 y 分桶和库存计数都进了特征。这意味着同样叫 mine_diamond，深层和浅层会得到不同的成功概率判断。

3.2 第二步：用注意力从案例库检索“同类状态”的历史证据

RADS 把查询向量 q 与案例库中每条案例向量做缩放点积注意力，得到权重 α ，再加权汇聚成上下文向量 ctx 。这里有两个为长序任务专门设计的约束：其一是同-waypoint 硬掩码——当库里同属 mine_diamond 的案例足够多时，注意力池被硬性限制在同子目标案例内，避免“砍木经验”污染“挖钻决策”；其二是 run_uuid 级防泄漏掩码——训练评估时同一次任务运行内的案例互相不可见，杜绝“拿同一局的后续步骤预测当前步”的信息泄漏。

映射到决策点 D：RADS 会重点关注库里那些“同样在深岩桶、同样持铁镐、同样接近钻石目标”的历史案例。如果这些案例里 dig_down 多次因岩浆失败、explore_horizontal 多次成功，注意力就会聚焦到它们身上。

3.3 第三步：对每个候选动作输出条件成功概率，并叠加一个稳健先验

把查询向量 q 、检索到的上下文 ctx 、候选动作嵌入三者拼接入决策头，sigmoid 输出该候选动作的条件成功概率 p_{success} 。这是和检索注入类、CBR 类最根本的分野——我的决策器吐出的是一个判别量（这个动作在这个状态下会不会成），不是“挑哪条经验”的检索权重，也不是交给大模型隐式权衡。

此外，针对长序任务里“稀有动作被高频动作带偏”的问题，RADS 在 logit 上叠加了一个 (waypoint, 动作) 的拉普拉斯平滑成功率先验作为残差（可训练权重）。举例：合成阶段的 craft_diamond_pickaxe 这种稀有动作，其嵌入容易被大量成功的泛化“craft”动作拉高；而该先验直接注入“这个具体 (waypoint, 动作) 组合的历史成功率”，把它拉回真实水平。这是一个小但实用的稳健性设计。

机制差异点 ②：判别式输出 + 可校准，而不是检索式或隐式

PRM 类也输出分数，但不带检索证据；CBR/记忆类把判断交给大模型，无法离线评估。

RADS 输出的 p_{success} 是可离线评估的：在留出集上可直接算 AUC、AP、F1、ECE（期望校准误差）。也就是说，“我的复用判断到底准不准、置信是否校准”这件事，在我的方法里是一个能拿数字回答的问题——这是审稿人最看重、而上述方法都给不出的东西。

3.4 第四步：低置信时弃权，把决策权交还给规划器

这是整套设计的灵魂。如果所有候选动作的 p_{success} 都低于阈值 $\min_p_{\text{success}}$ ，RADS 不强行选——它弃权，回退给视觉语言模型规划器从头规划。映射到决策点 D：假如智能体此刻进入了一个训练案例里从没出现过的诡异地形（比如深岩层里一个被岩浆三面包围的孤岛），RADS 会发现没有任何高置信的历史依据，于是主动放弃复用经验、让 VLM 重新审视局面，而不是硬套一条不可靠的历史动作把自己送进岩浆。

机制差异点 ③：会弃权、有回退通道——“知道自己何时不该决策”

检索注入类、CBR 类、PRM 类、Minecraft 记忆类——没有一个具备显式弃权 + 回退机制。它们在 D 点总会给出一个动作。

RADS 的弃权把“复用历史经验”和“从头规划”做成了两个可切换的决策通道，并由校准置信来路由。这既是安全性来源，也是算力分配机制：高置信时直接复用（省一次 VLM 调用），低置信才唤起规划器。

3.5 低层：当问题是“环境卡住”而非“选错动作”，Option 调度器接管

长序任务的失败常常不是高层选错，而是低层执行时被环境卡住——挖钻时陷进岩浆边、掉进空洞、被困在隧道死胡同。XENON-plus 把这类恢复能力封装成 option 技能（surface_escape 浮出水面、tunnel_recovery 隧道脱困等），由两件事协同：硬规则负责“当前环境下哪些恢复技能是合法/安全的”——只生成候选，相当于安全门；统计式 OptionDecisioner 按上下文签名分桶的历史成功率，在合法候选里排序选择。

最关键的是技能成败如何判定。它不看任务最终有没有成，而是看这次技能调用前后的环境状态差分（state delta）：tunnel_recovery 的成功被定义为“矿物增量达标或位移达标”，而非“最终合成出钻石镐”。映射到决策点 D 的下游：假如智能体挖钻时陷进一条死胡同，tunnel_recovery 被调用，只要它让智能体位移出了死胡同（位置变化达标），这次调用就记为成功并写入事件账本——哪怕整局任务后来因为别的原因失败了。

机制差异点 ④：技能级状态差分结算——把信用分配从任务终态解耦

PRM 用蒙特卡洛推演估“进展”，在 Minecraft 里跑一条到终点的续写代价极高；机器人恢复类（RACER/RecoveryChaining）多数仍以任务终态间接评价恢复行为。

我用环境可观测的状态差分直接测量“进展”，不需要任何推演。这可以论证为 PAV “进展即成功概率增量”思想在具身可观测环境下的零推演实现——把一个理论上昂贵的量，用环境本就提供的状态变化免费测出来了。同时它解决了长序任务的延迟混淆标签问题：一次有效的脱困不会因为整局后来失败而被错误地记为坏技能。

3.6 决策点 D 上的并排对照

把同一个决策瞬间上各方法的行为并排，机制差异一目了然（末行为本方法）。

方法	在 D 点检索/感知什么	吐出什么	标签/依据来源	错了怎么办
ExpeL / RAP	按相似度取历史轨迹片段+洞见，塞进 prompt	大模型隐式选一个动作	任务级成败（蒸馏为文本）	无弃权；硬给动作
DS-Agent (CBR)	相似度检索一条最像的方案案例	套用并由大模型 revise	外部分数（可验证）	靠大模型改写，无量化采纳判据
CBR-LLM 驾驶	按风险特征检索事故案例	用案例约束大模型决策	历史教训	案例当护栏，但与决策依据未分离
AgentPRM / PAV	不检索案例；看当前步状态	给每个候选动作打“进展”分	蒙特卡洛推演估成功率（Minecraft 中昂贵）	无弃权；分低也得选
Optimus-1 / JARVIS-1	检索“该任务以前怎么做成”的经验	大模型隐式选动作	任务级成败	无弃权；硬给动作
XENON（基线）	查 waypoint 动作计数表（不分状态）	选净成功最高的动作	waypoint 级计数	无置信无弃权；有成功就复用

方法	在 D 点检索/感知什么	吐出什么	标签/依据来源	错了怎么办
XENON-plus (本方法)	编码 52 维状态(含 y 分桶/库存), 注意力检索同 waypoint 同类状态案例	每个候选动作的条件成功概率 p_success (+证据案例)	子目标二值成败 + 技能级状态差分 (免推演)	低置信弃权→回退 VLM 规划器

表 1 决策点 D (深岩层挖钻、已持铁镐、diamond=2) 上各方法的运转对照

四、为什么是“开放世界长序任务”让我的设计拿到独有优势

一个方法的优势不能凭空声明，要扣回场景特点。开放世界长序任务有五个结构性特点，我的“案例库 + 决策器”恰好把每个特点转成了别的方法吃不到的红利。这一节是论文 motivation 的骨架。

4.1 特点一：奖励稀疏且延迟——只有任务终点才有信号

合成钻石镐要十余个子目标，只有最后成功才有明确回报。这让两类方法受困：依赖标量奖励做 Q 学习的 Memento 类要面对极稀疏信号；依赖推演估成功率的 PAV/PRM 类要把游戏跑到终点才能采样，代价高到不现实。

我的红利：子目标校验器给每个 waypoint 一个二值成败信号（这把铁镐做出来没有），把一条十余步的稀疏轨迹拆成十余个密集的监督样本；option 层的状态差分进一步在子目标内部给出技能级信号。于是 2600 条案例就能训出 AUC≈0.91 的决策器——这是“用环境天然的中间信号替代稀疏终点奖励”带来的直接好处，纯文本推理任务没有这种现成的中间可观测量。

4.2 特点二：状态高维且地形多变——“看起来相似”常常“实则不可迁移”

同样叫 mine_diamond，y=11 带岩浆的深岩层和 y=40 的安全矿洞，最优动作可能截然相反。按任务名或粗相似度检索的方法（Optimus-1、计数表 XENON）会把两者混为一谈，复用错误经验。

我的红利：状态条件化特征（尤其 y 分桶、库存计数）+ 同-waypoint 注意力检索，让决策器能区分地质环境。在 D 点，它检索的是“同样深岩、同样持铁镐”的经验，而不是所有 mine_diamond 经验的平均。这把“相似性”从任务级精确到了状态级。

4.3 特点三：失败高频且多为“环境卡死”——错因常在执行不在决策

长序任务里大量失败是低层被环境卡住（陷岩浆、掉空洞、困死胡同），而不是高层选错动作。只在高层做文章的方法（多数记忆/规划类）对这类失败无能为力。

我的红利：option 技能层 + 状态差分结算专门吃这块。脱困是否有效，用位移/矿物增量当场判定，不必等任务结束。这让“恢复能力”成为一个可统计、可调度、可累积的一等对象，而不是埋在大模型即时反应里的黑箱。

4.4 特点四：分布外情形不可避免——开放世界永远有没见过的局面

开放世界注定会遇到训练案例从未覆盖的地形组合。没有弃权机制的方法此刻最危险——它们会硬套一条最相似（但其实不可迁移）的历史经验。

我的红利：校准置信 + 弃权回退。当所有候选 p_success 都低，说明当前状态在案例库里没有可靠依据，决策器主动认怂、交还 VLM。这把“分布外检测”和“安全回退”内建进了决策流程——这是审稿人会特别认可的安全性论证，也是上述所有对照方法都不具备的。

4.5 特点五：高风险动作不可逆——掉进岩浆就重来

挖钻时一个错误的 `dig_down` 可能直接掉岩浆、丢掉整局进度。这要求决策既要会排序，更要有安全护栏。

我的红利：规则门控 + 决策器排序的解耦结构。硬规则先筛掉不安全候选（安全门），决策器只在合法候选里按成功概率排序，且低历史分不会禁用规则允许的候选（保守性）。相比把护栏与依据揉在一起的 CBR-LLM，这种“规则保安全、学习保优选”的分工更清晰、更易消融、也更易让审稿人相信系统不会因为决策器一时打分失误就做出灾难性动作。

一句话总结本章（可直接作为论文 contribution 的论证主线）

开放世界长序任务的五个特点——稀疏延迟奖励、高维多变状态、高频环境卡死、不可避免分布外、高风险不可逆——分别被我的设计转化为：中间二值/差分监督、状态条件化检索、技能级状态差分结算、校准弃权回退、规则门控解耦。其他方法至多覆盖其中一两点，没有任何一个方法在这五点上同时成立。

五、我的方法的不足，与针对性优化方案

要扛住拷打，必须先自己把刀递过去。下面每一条都先点出现状的薄弱处（审稿人很可能从这里下手），再给出优化方案和对应的可做实验。优化按“投入小、证据硬”到“更前沿、风险高”排序。

5.1 优化一（最该先做）：证明“案例库环”不是摆设——检索消融

薄弱处：审稿人会问——“你这 RADS 不就是个成功率分类器吗？把案例注意力去掉、只用当前状态特征预测 `p_success`，是不是一样好？要案例库干什么？”如果回答不了，整个“案例库 + 决策器”的双环叙事就垮一半。

优化方案：做一个去掉案例注意力的纯判别头变体（RADS-NoRetrieval，只吃 52 维状态特征 + 动作嵌入，不做检索），与完整 RADS 在同一留出集上比 AUC / AP / ECE。预期完整版显著更好，尤其在长尾 waypoint（如 `craft_diamond_pickaxe` 这种样本少的合成动作）和分布外状态上。这样就用数据证明了：检索环实质提升了决策环的判别质量与校准，二者不可分割。

用钻石镐例子说明这条消融为什么有力

在决策点 D，纯判别头只能依据“深岩+铁镐+diamond=2”这组特征给分，它学到的是全局平均规律。

完整 RADS 还能调出“上一次在几乎一模一样的深岩孤岛里 `dig_down` 掉了岩浆”这条具体案例，把 `dig_down` 的 `p_success` 压得更低。如果实验显示这种长尾/罕见状态上完整版的校准明显更好，就证明了案例检索带来的不是冗余，而是判别头单独学不到的、case-specific 的证据。

5.2 优化二：把弃权从“固定阈值”升级为“有保证的选择性决策”

薄弱处：`min_p_success` 现在是手设常数。审稿人会问——“阈值怎么定的？换个任务还成立吗？有没有理论支撑？”一个魔法数字很容易被攻击。

优化方案：引入校准 + 选择性预测的形式化。其一，用温度缩放或 Platt 标定把 `p_success` 校准好（报告 ECE 前后对比），让“0.7 就是真的七成会成”。其二，更进一步，用保形预测（conformal prediction）为“复用此动作”给出风险上界，把弃权准则变成“当且仅当风险上界超过容忍度时回退”，从而获得分布无关的覆盖保证。这样弃权就从“我设了个阈值”升级为“我形式化了一个带保证的 reuse-or-replan 决策规则”，同时把我和 Memento（无弃权）、PRM（无回退）彻底区分开。

配套实验：画弃权率—回退收益曲线（横轴弃权阈值，纵轴任务成功率与平均 VLM 调用次数），展示存在一段区间，弃权既提升成功率又不显著增加规划开销——这正好呼应特点四与特点五。

5.3 优化三：把 state-delta 锚到 PAV 的“进展”理论，并做对照

薄弱处：状态差分判定技能成败，现在读起来像一个工程 trick（“我定义矿物增量达标就算成功”），缺理论身份，审稿人可能觉得 ad-hoc。

优化方案：在方法论里显式把它定义为“进展奖励的零推演具身实现”。PAV 证明了步骤价值 = 成功概率增量（进展），但要靠采样推演估计；我主张在状态可观测的具身环境里，进展可以用环境状态差分直接、无偏、零成本地测量。把 tunnel_recovery 的成功定义写成一个进展函数 $\Delta(s_{\text{before}}, s_{\text{after}})$ 的阈值化，并讨论它与 PAV 进展信号的对应与差异（我的是确定性可观测测量，PAV 的是蒙特卡洛估计量）。配一个小实验：对比“状态差分结算”与“任务终态归因”两种标签训练出的 option 调度器，证明前者带来的技能选择更准、更不受长程混淆影响。

5.4 优化四（更前沿）：让决策器反过来驱动案例采集——主动经验闭环

薄弱处（也是机会）：现在数据流是单向的——案例训练决策器。但决策器的弃权/低置信信号其实蕴含了宝贵信息：它在告诉你“案例库在哪些状态区域是空的”。

优化方案：把高弃权率的状态区域（如某种罕见地质组合）标记为“案例稀疏区”，在后续运行中引导智能体优先去这些区域探索补案例，形成“决策器指认盲区→主动采集→回填案例库→决策器变强”的主动经验获取闭环。这一思路在自进化记忆一脉（Memory-R1、MemRL）刚起步，而结合具身环境免推演标签是很新的点，可作为论文的前瞻贡献或未来工作。需权衡：会增加系统复杂度与实验量，建议视投稿时间决定是落地还是仅作 discussion。

5.5 优化五：补齐与“隐式复用”的正面对照基线

薄弱处：要证明“显式判别 > 隐式判断”，光和自家消融比不够，得有一个代表隐式路线的强基线。

优化方案：实现一个 ExpeL/RAP 式基线——同样的案例库，但复用方式改为“检索 top-k 案例注入 VLM 上下文、由 VLM 隐式决定动作”，不用 RADS 打分。在同一批长序任务上比成功率与单步决策延迟。预期 RADS 在成功率上相当或更高，但在可解释性（能给出 p_{success} 与证据案例）和决策成本（高置信时省去 VLM 调用）上明显占优。这条基线直接支撑“为什么要做显式决策器而不是继续塞 prompt”这一核心质疑。

5.6 优化方案与对应实验一览

优化	针对的审稿质疑	关键实验/产出	优先级
①检索消融	“去掉案例库是不是一样？”	RADS vs RADS-NoRetrieval 的 AUC/ECE，长尾与OOD分层	最高
②选择性决策	“阈值是魔法数字”	校准(ECE↓)+保形风险界；弃权率—收益曲线	高
③进展理论锚定	“状态差分太 ad-hoc”	定义进展函数 Δ ；差分结算 vs 终态归因 的调度准确率	高
④主动经验闭环	“数据流单向、被动”	弃权指认盲区→主动采集→AUC随轮次上升曲线	中(可选)
⑤隐式基线对照	“为什么不继续塞 prompt？”	ExpeL式隐式复用基线 vs RADS：成功率/延迟/可解释性	高

表 2 五条优化方向、其针对的审稿质疑与对应实验

六、可写进论文的“区分性陈述”与审稿问答预案

6.1 一句话区分性陈述 (建议放进 intro 与 related work 收尾)

Positioning Statement

不同于把经验注入上下文交由大模型隐式裁决的方法 (ExpeL、RAP、Optimus-1、JARVIS-1)，也不同于输出步骤分数但不检索证据、不具弃权、且依赖昂贵推演估计进展的过程验证器 (PAV、AgentPRM)，本文的决策器对“历史经验在当前状态下的可迁移性”输出可校准的条件成功概率，以证据案例支撑、在低置信时弃权并回退规划器；并以环境状态差分对技能进行免推演的进展结算，从而把信用分配与任务终态解耦。这一组合专为开放世界长序任务的稀疏延迟奖励、高维多变状态、高频环境卡死、不可避免的分布外与高风险不可逆五个特点设计，现有方法均未在此五点上同时成立。

6.2 审稿人最可能的六个问题与应答

Q：这不就是一个成功率分类器吗？案例库是不是多余？

答：检索消融（优化一）直接回应。完整 RADS 与去掉案例注意力的纯判别头相比，在长尾子目标与分布外状态上的 AUC/ECE 明显更好——证明案例检索提供的是判别头单独学不到的 case-specific 证据，双环不可分割。

Q：和 Memento（学习案例检索策略）有什么本质区别？

答：Memento 学的是“挑哪条案例”的检索分布，依赖环境标量奖励做 Q 学习，且总会挑一条交给 LLM、不弃权。我学的是“这个动作在这个状态会不会成”的条件成功概率，监督来自子目标二值成败（无需 reward shaping），且低置信时弃权回退。输出对象、监督信号、弃权机制三者都不同。

Q：和 PRM/PAV 这类过程验证器比，新意在哪？

答：三点。其一，我检索历史案例并给出证据，PRM 只给分；其二，我有弃权回退通道，PRM 分低也得选；其三，最关键，PAV 估“进展”要蒙特卡洛推演，在 Minecraft 里要把游戏跑到终点、代价极高，而我用环境状态差分零推演地直接测进展。我可被视为 PAV 进展思想在具身可观测环境下的免推演特例。

Q：弃权阈值是怎么定的？会不会是为了好看调出来的？

答：优化二回应。我们对 p_{success} 做校准（报告 ECE），并用保形预测给出分布无关的风险界，把弃权准则形式化为带覆盖保证的决策规则，而非单一魔法阈值；并给出弃权率—收益曲线展示其鲁棒区间。

Q：状态差分判定技能成败是不是太随意（ad-hoc）？

答：优化三回应。我们把它定义为进展函数 $\Delta(s_{\text{before}}, s_{\text{after}})$ 的阈值化，并在理论上对应到 PAV 的进展信号（差异在于我的是确定性可观测测量、PAV 是蒙特卡洛估计量），同时用“差分结算 vs 任务终态归因”的对照实验证明前者带来的技能选择更准、抗长程混淆。

Q：为什么不直接把案例塞进 VLM 上下文，省得训练一个决策器？

答：优化五回应。我们实现了 ExpeL 式隐式复用基线（同样案例库、改由 VLM 隐式裁决）。RADS 在成功率上相当或更优，且能输出 p_{success} 与证据案例（可解释、可审计），并在高置信时省去 VLM 调用（更省算力）。显式决策器换来的是隐式路线给不了的可评估性、安全弃权与成本可控。

七、结语：把“组合优势”讲成“必须如此组合的理由”

审稿人最常见的攻击不是“你某个零件不新”，而是“你这些零件别人都有，凭什么拼在一起就算贡献”。本报告给出的应对是：把每个机制点扣回开放世界长序任务的一个具体结构性特点（第四章），再用消融与对照证明去掉任一环都会变差（第五章）。这样，“案例库 + 决策器”就不再是若干现成技术的拼装，而是为这一类任务量身定制、且每个部件都有不可替代理由的统一方法。建议落笔顺序：先用合成钻石镐的决策点 D 作为 running example 贯穿全文（intro 抛出、method 拆解、experiment 回扣），让审稿人始终有一个具体画面；再以第六章的区分性陈述与问答预案逐条堵住质疑入口。

说明：本报告基于 XENON-plus 代码实现与 2024 年以来公开文献综合撰写，所有对比方法的描述为机制层转述；涉及的实验为建议方案，预期结论需以实际运行为准。